

PRAKTIKUM SISTEM OPERASI

MODUL 11

MEMORI XINU



Oleh:

Rizaldy Aulia Rachman 2311104051

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK

DIREKTORAT KAMPUS PURWOKERTO

UNIVERSITAS TELKOM

2026

JURNAL

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

```
#include <xinu.h>

// Nama: Rizaldy Aulia Rachman
// NIM: 2311104051

shellcmd xsh_freememory(int nargs, char *args[]) {

    struct memblk *blk;
    uint32 totalFreeSpace = 0;

    kprintf("Free List:\n");
    kprintf("Block address Length (dec) Length (hex)\n");
    kprintf("-----\n");

    blk = memlist.nnext;

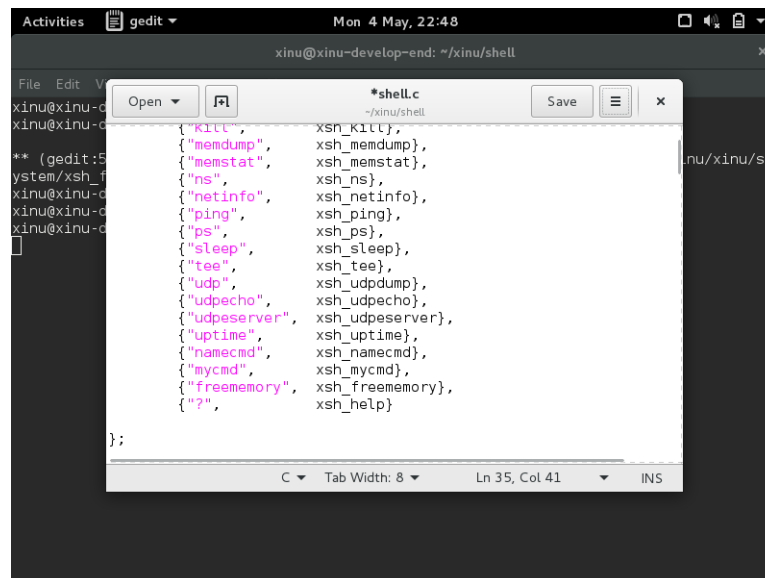
    while (blk != NULL) {
        kprintf("0x%08X %10d 0x%08X\n", (uint32) blk, blk->nlength, blk->nlength);

        totalFreeSpace += blk->nlength;
        blk = blk->nnext;
    }

    kprintf("\nTotal FreeSpace %d\n", totalFreeSpace);

    return 0;
}
```

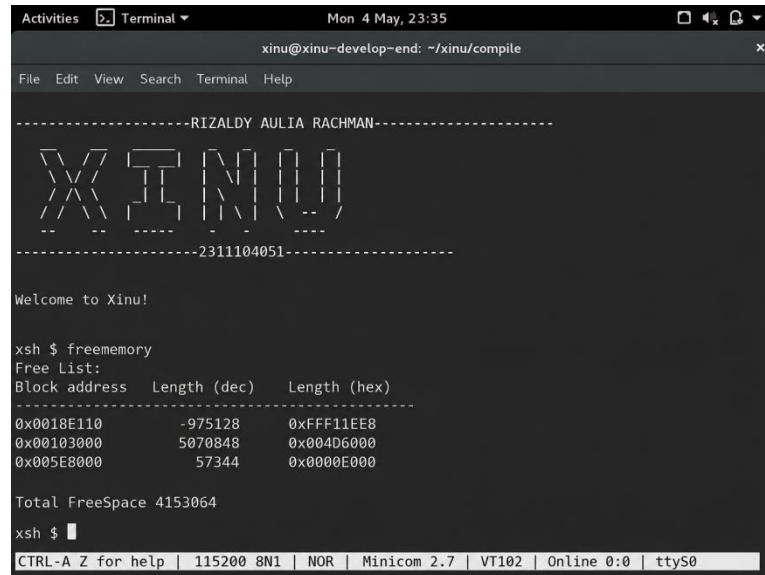
Edit shell.c



Edit shprototypes.h

```
/* in file xsh_freememory.c */
extern shellcmd xsh_freememory(int32, char *[]);
```

Output



```
xinu@xinu-develop-end: ~/xinu/compile
-----RIZALDY AULIA RACHMAN-----
XINU
-----2311104051-----

Welcome to Xinu!

xsh $ freememory
Free List:
Block address      Length (dec)      Length (hex)
-----
0x0018E110         -975128          0xFFFF11EE8
0x00103000          5070848          0x004D6000
0x005E8000           57344           0x0000E000

Total FreeSpace 4153064
xsh $
```

2. Jawablah pertanyaan berikut:

- a. Mengapa Xinu memisahkan data segment dan BSS segment?

Xinu memisahkan Data Segment dan BSS Segment karena keduanya memiliki fungsi yang berbeda. Data Segment digunakan untuk menyimpan variabel global yang telah diberi nilai awal, sedangkan BSS Segment digunakan untuk menyimpan variabel yang belum diinisialisasi. Dengan pemisahan ini, penggunaan memori menjadi lebih efisien karena isi BSS tidak perlu disimpan secara penuh di dalam file program.

- b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?

Alokasi memori menyebabkan jumlah *free space* berkurang karena sebagian memori digunakan oleh proses yang sedang berjalan. Sebaliknya, saat terjadi dealokasi, memori akan dikembalikan ke sistem sehingga *free space* bertambah. Akibatnya, ukuran *free space* dapat berubah-ubah selama proses eksekusi di Xinu berlangsung.

- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?

Heap memiliki risiko yang lebih besar karena pengelolaan memorinya dilakukan secara manual. Jika tidak dikelola dengan benar, dapat terjadi *memory leak* maupun fragmentasi memori. Sebaliknya, stack dikelola secara otomatis menggunakan mekanisme LIFO (*Last In, First Out*), sehingga umumnya lebih aman dan mudah dikendalikan.

- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
Xinu menggunakan struktur *linked list* karena lebih fleksibel dalam mengelola blok memori kosong dengan ukuran yang berbeda-beda. Struktur ini juga mempermudah proses pencarian, penambahan, serta penggabungan kembali blok memori yang tersedia secara dinamis.
- e. Apa tantangan utama dalam penggunaan heap di Xinu?
Tantangan utama dalam penggunaan heap di Xinu adalah risiko terjadinya *memory leak*, fragmentasi memori, serta kebutuhan pengelolaan secara manual yang lebih rumit dibandingkan stack. Kondisi tersebut dapat menyebabkan penggunaan memori menjadi kurang optimal dan tidak efisien.